Fourth Down Engine — technical documentation

Version: 1.0.0

This documentation tells you how to install, run and maintain the engine. It uses ASD-STE100 Simplified Technical English. It does not give results. For results, figures and method, read the white paper: fourth-down-whitepaper.md.

Type	Document	Use it to
Tutorial	tutorial/getting-started.md	Open the app and get your first call.
How-to	how-to/use-the-app.md	Set a situation and the factors.
How-to	how-to/rebuild-everything.md	Rebuild the models, grades and documents from raw data.
How-to	how-to/update-for-a-new-season.md	Add a season or a rule change.
How-to	how-to/add-a-factor.md	Add a new input to the engine.
Reference	reference/files-and-commands.md	Find each file, command and output.
Reference	reference/engine-api.md	Call the engine from Python or JavaScript.
Reference	reference/config.md	Change a setting.
Explanation	explanation/how-the-engine-decides.md	Understand the parts and why they exist.

Decisions: adr/. Risks: ../RISK_REGISTER.md. Generated reports: reports/.

Tutorial: get your first fourth-down call

This tutorial takes 5 minutes. You need a web browser. You do not need Python.

1. Open the app

1. Double-click **Fourth Down Engine** on the desktop.
2. If the shortcut is not there, open `app/index.html` in the project folder.

The app opens on the **Decision** tab. It shows a sample situation: down 3, 4:30 left in the fourth quarter, 4th & 3 at the opponent's 34.

Click **Get the call**.

2. Read the call

1. Look at **The call**. It shows the best option in large letters.
2. Read the line below it. It gives the edge in win-probability points.
3. Look at the **split** bar. It shows how the 20 resampled models vote among the options. It adds up to 100%.
4. Look at the bars under **Win chance after each choice**. Each bar is your chance to win after that choice.

3. Change the situation

1. Click **Edit situation**.
2. Set **Yards to go** to 8. Click **Get the call**. The call can change to a field goal.
3. Click **Edit situation**. Set **Clock** to 0 min, 4 sec. Click **Get the call**. With 4 seconds left and down 3, only the field goal ties.
4. Click **Edit situation**. Click **2nd** down. Click **Get the call**. With 4 seconds left, the app tells you to kick now, not run a play.

4. Turn on a factor

1. Click **Edit situation**. Turn on **Weather**.
2. Move **Wind** to 25 mph. Click **Get the call**.
3. Look at the field-goal bar. Wind makes a long kick less likely.

5. Read the map

1. Look at the **Decision map**. Each square is one spot and one distance, with your score, clock and factors.
2. Green is go, gold is field goal, blue is punt. A faded square is a toss-up.

3. Tap a square. The app loads that situation and gives its call.

Next

- Read ../how-to/use-the-app.md for every control.
- Open the **Coaches** and **Kickers** tabs for the grades.

How to add a factor

This example adds a new field-goal input. Other inputs follow the same steps.

1. In `engine/load_data.py`, derive the new column from the play-by-play.
2. In `engine/fit_models.py`, add the column to the frame (`fg_frame`) and to the feature lists (for example `FG_WEATHER`).
3. In `engine/decide.py`, add the input to the feature dictionary in `p_field_goal`. Give it a neutral value for when its toggle is off.
4. In `app/engine.js`, make the same change. The two files must stay the same in logic.
5. In `app/index.html`, add a control and read it in `readState`.
6. Add a test in `tests/test_decide.py` with a value that you calculate by hand.
7. Rebuild everything. The parity test fails if step 3 and step 4 differ.
8. Write a CHANGELOG entry. A new factor that moves a published figure is a MINOR change.

How to rebuild everything from raw data

You need Python 3.12 and Node.js 20 or later.

1. Install the Python packages:

```
bash py -m pip install pandas pyarrow numpy scikit-learn pytest weasyprint markdown python-pptx
```

2. Download the raw data (about 250 MB):

```
bash py engine/download_data.py
```

3. Fit the models (about 1 minute). This writes `app/model.js`, `data/model.json`, `data/validation.json` and `data/kickers.json`:

```
bash py engine/fit_models.py
```

4. Fit the bootstrap replicates (about 5–20 minutes). This writes `app/bootstrap.js`:

```
bash py engine/bootstrap.py
```

5. Grade the coaches (about 10 minutes). This writes `data/coach_grades.json` and `app/grades.js`:

```
bash py engine/grade_coaches.py
```

6. Make the weekly page and the edge check:

```
bash py engine/weekly.py py engine/edge_check.py
```

7. Render the reports and the results block in the white paper:

```
bash py build/render_reports.py
```

8. Build the PDFs and the deck:

```
bash py build/build_docs.py
```

9. Run the tests. All tests must pass:

```
bash py -m pytest tests -v
```

Do the steps in this order. Each step reads the output of the step before it. If you change the model (step 3), do steps 4 to 9 again.

How to update for a new season or a rule change

Add a completed season

1. Open `engine/config.py`.
2. Add the season to `seasons`. Add it to `grade_seasons` if it is not there.
3. Move `holdout_seasons` to the two newest complete seasons.
4. Set `kicker_list_seasons` to the two newest complete seasons.
5. Set `current_season` to the season in progress.
6. Increase `version` (see the rules below).
7. Do all steps in rebuild-everything.md.

Refresh the current season during the year

The **Weekly calls** GitHub Action does this every Tuesday: it runs `py engine/weekly.py --download`, commits `app/weekly.js` and redeploys the site. To do it by hand, or to refresh the coach grades too:

1. Download the new weeks:

```
bash py engine/download_data.py --force
```

2. Do steps 5 to 9 in rebuild-everything.md. You do not refit the model.

Apply a rule change

1. If the change moves the kickoff touchback, set `rules_season` to the first season under the new rule. The engine measures the new kickoff spot from that season.
2. If the change moves the missed-field-goal spot, change `fg_snap_to_spot` or `missed_fg_min_spot`.
3. Rebuild everything.

Version and changelog

- No published figure moves: PATCH.
- A figure moves and the method stays the same: MINOR.
- The method changes, so old and new figures answer different questions: MAJOR.

Write one `CHANGELOG.md` entry with a `### Record` section in the same pass.

How to use the app

Fill from a live game (website only)

1. Click **Fill from a live game**. The app loads this week's games from ESPN.
2. Pick the game. Games in progress are at the top, marked LIVE.
3. Tap your team. The two buttons show the teams in the game you picked.
4. The app fills in the teams, betting line and roof, and in a live game the score, clock, down, distance, ball spot and timeouts.
5. Check every field. The feed is unofficial and can be late or wrong. Then click **Get the call**.

Set the situation

1. Optional: pick **Our team** and **Their team** under **Matchup**. The app sets the stadium (roof, altitude) from the home team and your kicker from your team. You can still change them.
2. Enter **Our score** and **Their score**.
3. Click the **Quarter**.
4. Enter the **Clock** (minutes and seconds left in the quarter). The app shows if the two-minute warning is still to come.
5. Click **Own** or **Opp** and set the yard line (1–50).
6. Click the **Down**. The app starts on 4th down.
7. Set **Yards to go**. On 4th & 1, click how far it is: **Inches**, **Not sure** or **About a yard**.
8. Click the number of timeouts for each team.
9. Under **Where's the game?**, click **Our stadium**, **Their stadium** or **Neutral site** (London, Mexico City, the Super Bowl). It sets home-field advantage and, with a matchup picked, the stadium.
10. In the first half, click who gets the second-half kickoff.
11. Click **Get the call**. The app shows the results screen. To change the situation, click **Edit situation**. Your inputs stay.

Use the **−** and **+** buttons to change a number by 1. Hold a button to repeat. The seconds and the yard line also have **−5** and **+5** buttons. The app calculates only when you click **Get the call**.

On 1st, 2nd or 3rd down the app compares **Run a play** with **Kick a FG now**. Use this when the clock is about to run out.

Set the factors

Each factor has a switch. A factor that is off uses a neutral value.

Factor	When on, set	When off, the engine uses
Weather	Wind (mph), temperature (°F), rain or snow	No wind, 70 °F, dry

Factor	When on, set	When off, the engine uses
Stadium	Roof (outdoors or dome), altitude	An average stadium
Team strength	Pre-game point spread, as a betting line (−7 = you are 7-point favorites)	Equal teams, with the usual home-field edge for the home team
Kicker	A kicker, league average, or replacement level	A league-average kicker

A dome cancels weather. The kicker list shows kickers with at least 10 attempts in the last two seasons.

Share a call

1. On the results screen, click **Share this call**.
2. On a phone, pick an app to send it. On a computer, the link is copied; paste it in a message.
3. The link opens the app at the same situation and shows the call. It is short: the situation is packed into about 21 digits.

Put the app on your home screen

- iPhone (Safari): tap **Share**, then **Add to Home Screen**.
- Android (Chrome): tap the menu, then **Add to Home screen** or **Install app**.

Read the output

The first bar is the **split**: how the 20 resampled models vote among the options. It adds up to 100%. For example, "Go for it 95% · Field goal 5% · Punt 0%" means 19 of 20 models pick going for it.

The bars under **Win chance after each choice** show how often you win the game if you choose each option. They are separate "what ifs" and do not add up to 100%.

- **The call**: the option with the highest win probability. The line below it says why, in one sentence. "Toss-up" means the edge is under 1 point.
- **The split**: 90% or more for one option is a confident call; 70–85% is a lean; less is genuinely uncertain.
- **Detail boxes**: the odds and win probability behind each option.
- **Decision map**: the call at every spot and distance for the current score, clock and factors.

See every game

1. Click **Games**. Pick a week at the top. The newest week opens first.
2. Each game shows both teams with a grade: **A+** means no confident mistakes; otherwise the team is ranked against every team-game since 2014 that had one.
3. Tap a game to see every fourth down: the coach's call, the engine's call, and what happened.
4. Under the games: the week's costliest calls, the best gutsy calls, and the season so far by team.

See the grades

1. Click **Coaches**. The switch at the top picks the seasons: **Since 2021** (the default) grades only recent calls, so a long career is not judged by an older era's norms; **All since 2014** grades every call. Click a column name or use **Sort by** to sort. Click a coach to see his seasons and costliest calls.
2. Click **Kickers**. The table shows current kickers and the replacement level.

A **confident mistake** is a call where at least 18 of the 20 resampled models agree the coach chose wrong. **Wins lost per season** adds up the win chance those mistakes gave away, scaled to a 17-game season. For more, read the white paper, section 5.

Reference: configuration

All settings that change over time are in one block: `CONFIG` in `engine/config.py`. After you change a setting, rebuild (see ../how-to/rebuild-everything.md).

Setting	Meaning
`version`	Engine version. It must equal the newest CHANGELOG entry.
`seasons`	Seasons used to fit the models.
`holdout_seasons`	Seasons held out for validation.
`rules_season`	Season that sets the kickoff spot (current kickoff rules).
`touchdown_points`	Points for a touchdown before the try (6).
`try_seasons_from`	First season used to measure extra-point and 2-point rates.
`short_yardage`	4th & 1 true distances (Lopez 2020): average, went, kicked, inches, full yard.
`fg_snap_to_spot`, `fg_distance_add`, `missed_fg_min_spot`	Field-goal geometry and the missed-kick rule.
`fg_max_distance`	Longest kick the engine allows.
`tossup_margin`	Edge below which a call is a toss-up.
`fair_catch_kick_window`	Seconds left after a punt at or below which the opponent may use a fair-catch kick.
`wp_trees`	Win-probability model settings.
`kicker_prior_grid`	Candidate prior widths for kicker accuracy and range.
`replacement`, `replacement_fringe`	Replacement-kicker rules (main and sensitivity).
`kicker_list_seasons`, `kicker_list_min_attempts`	Which kickers the app lists.
`kicker_grade_min_attempts`	Minimum attempts for a kicker letter grade.
`precip_words`, `altitude_stadiums`, `cold_threshold_f`	Weather and stadium definitions.
`quantiles`, `togo_buckets`, `punt_bin_width`	Outcome-distribution settings.
`grade_seasons`, `current_season`	Seasons graded; the season in progress.
`coach_aliases`	Corrections to coach names in the source data.
`grade_min_games`	Minimum games for a coach letter grade.
`grade_toggles`	Factors used when grading coaches.
`bootstrap_models`, `bootstrap_seed`	Number of bootstrap replicates; random seed.
`confident_share`	Share of replicates needed for a confident mistake.
`bootstrap_draws`	Resamples for each coach's interval.

Reference: engine API

The Python engine (`engine/decide.py`) and the JavaScript engine (`app/engine.js`) have the same functions and give the same numbers.

decide(model, state, toggles)

Returns the win probability of each option and the call.

state (all numbers):

Key	Meaning
`score_diff`	Our score minus their score.
`qtr`	Quarter, 1–4.
`half_seconds`	Seconds left in the half (1–1800).
`yardline`	Yards to the opponent's goal line (1–99).
`ydstogo`	Yards to go (1 or more, not more than `yardline`).
`down`	1–4. Default 4. On 1–3 the options are a play (`go`) and a field goal; `punt` is null.
`pos_to`, `def_to`	Our timeouts, their timeouts (0–3).
`home`	1 if we are the home team, 0 if away, 0.5 at a neutral site.
`receive_2h`	1 if we receive the second-half kickoff (first half only).
`spread`	Points we are favored by (team-strength factor).
`wind`, `temp`, `precip`	mph, °F, 0 or 1 (weather factor).
`indoor`, `altitude_kft`	0 or 1; thousands of feet (stadium factor).
`kicker_a`, `kicker_b`	Kicker accuracy and range (kicker factor).

toggles: `{weather, stadium, spread, kicker}`, each true or false.

Returns:

```
{ wp: {go, fg, punt},        // fg is null when the kick is over 70 yd
  best: "go" | "fg" | "punt",
  margin: number,            // best minus second best, in WP (0-1)
  tossup: boolean,           // margin < tossup_margin
  detail: { go: {p_convert, wp_if_convert, wp_if_fail},
            fg: {p_make, wp_if_make, wp_if_miss, distance},
            punt: {opp_yardline_mean, p_return_td, p_muff, fair_catch_kick_risk} } }
// on downs 1-3, detail.go is {p_convert (first down), wp_if_convert, wp_if_fail, p_turnover}
```

Other functions

Python / JavaScript	Returns
`wp_raw` / `wpRaw`	WP for the team with the ball in snap state `s`.
`p_convert` / `pConvert`	Conversion probability.
`p_field_goal` / `pFieldGoal`	Make probability.
`punt_spots` / `puntSpots`	Opponent start spots after a normal punt (20 quantiles).

Example (Python)

```
import json, sys
sys.path.insert(0, "engine")
import decide
model = json.load(open("data/model.json"))
state = dict(score_diff=-3, qtr=4, half_seconds=270, yardline=34, ydstogo=3,
             pos_to=2, def_to=3, home=1, receive_2h=0)
print(decide.decide(model, state))
```

Example (Node)

```
const M = require("./app/model.js"), E = require("./app/engine.js");
console.log(E.decide(M, { score_diff: -3, qtr: 4, half_seconds: 270, yardline: 34,
  ydstogo: 3, pos_to: 2, def_to: 3, home: 1, receive_2h: 0 }, {}));
```

Reference: files and commands

Folders

Folder	Contents
`app/`	The app. `index.html` (page), `engine.js` (engine), `model.js`, `bootstrap.js`, `grades.js` (generated data).
`engine/`	Python pipeline: config, data loading, model fitting, bootstrap, coach grading, reference engine.
`build/`	`grade.js` (scores decisions in Node), `render_reports.py`, `build_docs.py`, generated PDFs and deck.
`data/`	Generated JSON. `raw/` holds the downloaded parquet files (not in git).
`docs/`	White paper, deck source, this documentation, ADRs, generated reports.
`tests/`	pytest suite.
`assets/`	App icon.

Commands

Command	Reads	Writes	Time
`py engine/download_data.py [--force]`	GitHub (nflverse)	`data/raw/*.parquet`	1 min
`py engine/fit_models.py`	`data/raw/`	`app/model.js`, `data/model.json`, `data/validation.json`, `data/kickers.json`	1 min
`py engine/bootstrap.py`	`data/raw/`, `data/model.json`	`app/bootstrap.js`	5–20 min
`py engine/grade_coaches.py`	`data/raw/`, `app/model.js`, `app/bootstrap.js`, `data/kickers.json`	`data/coach_grades.json`, `app/grades.js`, `data/decisions.csv.gz`	~10 min
`py engine/weekly.py [--download]`	`data/raw/` (current season), `app/model.js`, `app/bootstrap.js`	`data/weekly.json`, `app/weekly.js`	1 min
`py engine/edge_check.py`	`data/decisions.csv.gz`	`data/edge_check.json`	1 s
`py build/render_reports.py`	`data/*.json`	`docs/reports/*.md`, results block in the white paper	1 s
`py build/build_docs.py`	`docs/*.md`	`build/*.pdf`, `docs/fourth-down-deck.pptx`, `docs/fourth-down-deck.md`	30 s

Command	Reads	Writes	Time
`py -m pytest tests -v`	`app/model.js`, `engine/`, `data/*.json`	—	20 s

Generated files — do not edit by hand

`app/model.js`, `app/bootstrap.js`, `app/grades.js`, `app/weekly.js`, `data/*.json`, `data/decisions.csv.gz`, `docs/reports/*.md`, `docs/fourth-down-deck.md`, and the block between the `GENERATED RESULTS` markers in the white paper.

Explanation: how the engine decides

This page explains the parts of the engine and why each part exists. For the equations and the results, read the white paper.

One question for every option

The engine asks one question for each option: after this choice, what is our chance to win the game? It does not use points. Points ignore the score and the clock. A field goal that ties the game with 4 seconds left is worth more than 3 points.

Parts

1. **Win-probability model.** It gives the chance to win from any snap. It learns from every play from 2014 to 2025. All other parts feed it.
2. **Conversion model.** It gives the chance to make the first down. It learns from 3rd and 4th downs together, because 4th-down attempts alone are few and biased.
3. **Field-goal model.** It gives the chance to make a kick from its distance, the weather, the stadium and the kicker.
4. **Punt model.** It gives where the opponent starts after a punt, from real punts from the same part of the field.
5. **Clock.** It moves the clock by the real time each kind of play takes. It stops the clock at the two-minute warning. It ends the half or the game.

Why toggles

Each optional factor can be off, so you can see what it changes. Off does not mean zero. Off means "an average situation for this factor".

Why a bootstrap

The models learn from a finite number of games. A different sample of games gives slightly different models. The engine fits 20 models on resampled games. If most of them agree, the call is confident. If they disagree, the call is a toss-up, and a coach who chose differently did not make a mistake.

Why two engines

The app runs in a browser, so it uses JavaScript. The pipeline uses Python. The two engines do the same calculation. A test in CI checks that they give the same numbers.